

TurnosRed

Informe técnico - Actividad 1

Asignatura: Integraciones Web

Fecha de ejecución: 3 de septiembre de 2026

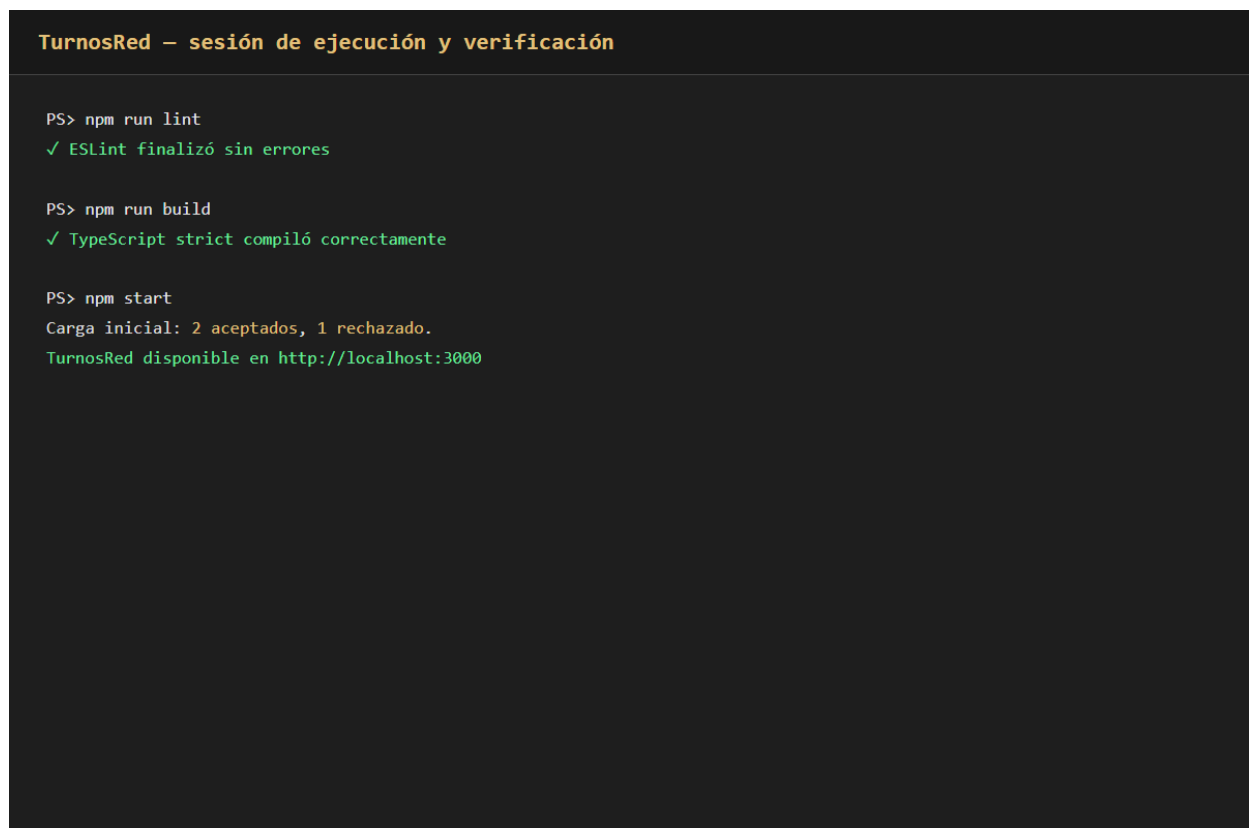
Backend profesional para normalizar, administrar y comunicar en tiempo real turnos médicos provenientes de fuentes JSON heterogéneas.

1. Solución implementada

Para resolver la actividad partí del proyecto base y lo reorganizé por responsabilidades. Separé los modelos, la lógica de negocio, los controladores, las rutas y los eventos para que cada parte pueda entenderse y probarse de manera independiente.

La lectura inicial utiliza **node:fs/promises** con `async/await` y `try/catch`. Durante la carga se limpian los nombres, se convierten los identificadores y documentos, se unifican fecha y hora, y se interpretan las distintas formas de indicar si un turno está confirmado. Los registros incompletos o inválidos no ingresan al sistema.

Captura de la ejecución inicial

A terminal window with a dark background and light green text. The title bar reads "TurnosRed – sesión de ejecución y verificación". The terminal shows three commands being executed: "npm run lint", "npm run build", and "npm start". The output for "npm start" shows "Carga inicial: 2 aceptados, 1 rechazado." and "TurnosRed disponible en http://localhost:3000".

```
TurnosRed – sesión de ejecución y verificación

PS> npm run lint
✓ ESLint finalizó sin errores

PS> npm run build
✓ TypeScript strict compiló correctamente

PS> npm start
Carga inicial: 2 aceptados, 1 rechazado.
TurnosRed disponible en http://localhost:3000
```

Captura 1. Compilación, control de estilo y carga inicial del servidor.

La salida confirma que dos registros fueron normalizados y que el registro con identificador inválido fue rechazado. La configuración `.vscode/launch.json` permite repetir la depuración paso a paso.

2. Pruebas de la API REST

Los cinco endpoints se ejecutaron contra el servidor local. La colección importable se encuentra en postman/TurnosRed.postman_collection.json y los resultados completos en evidencias/resultados-api.json.

Prueba	Método	Estado	Tiempo
Obtener todos	GET	200	5 ms
Obtener por ID	GET	200	3 ms
Crear turno	POST	201	32 ms
Actualizar turno	PUT	200	14 ms
Eliminar turno	DELETE	200	3 ms

Resultado general: APROBADO. Los cinco recorridos principales respondieron con el código esperado. La colección preparada puede importarse en Postman para repetir cada solicitud.

Captura de las pruebas

TurnosRed – ejecución de pruebas HTTP

```
PS> node scripts/generar-evidencias.mjs
```

Prueba	Método	Estado	Duración
Obtener todos	GET	200	5 ms
Obtener por ID	GET	200	3 ms
Crear turno	POST	201	32 ms
Actualizar turno	PUT	200	14 ms
Eliminar turno	DELETE	200	3 ms

Resultado general: APROBADO
5 de 5 solicitudes respondieron con el código HTTP esperado.

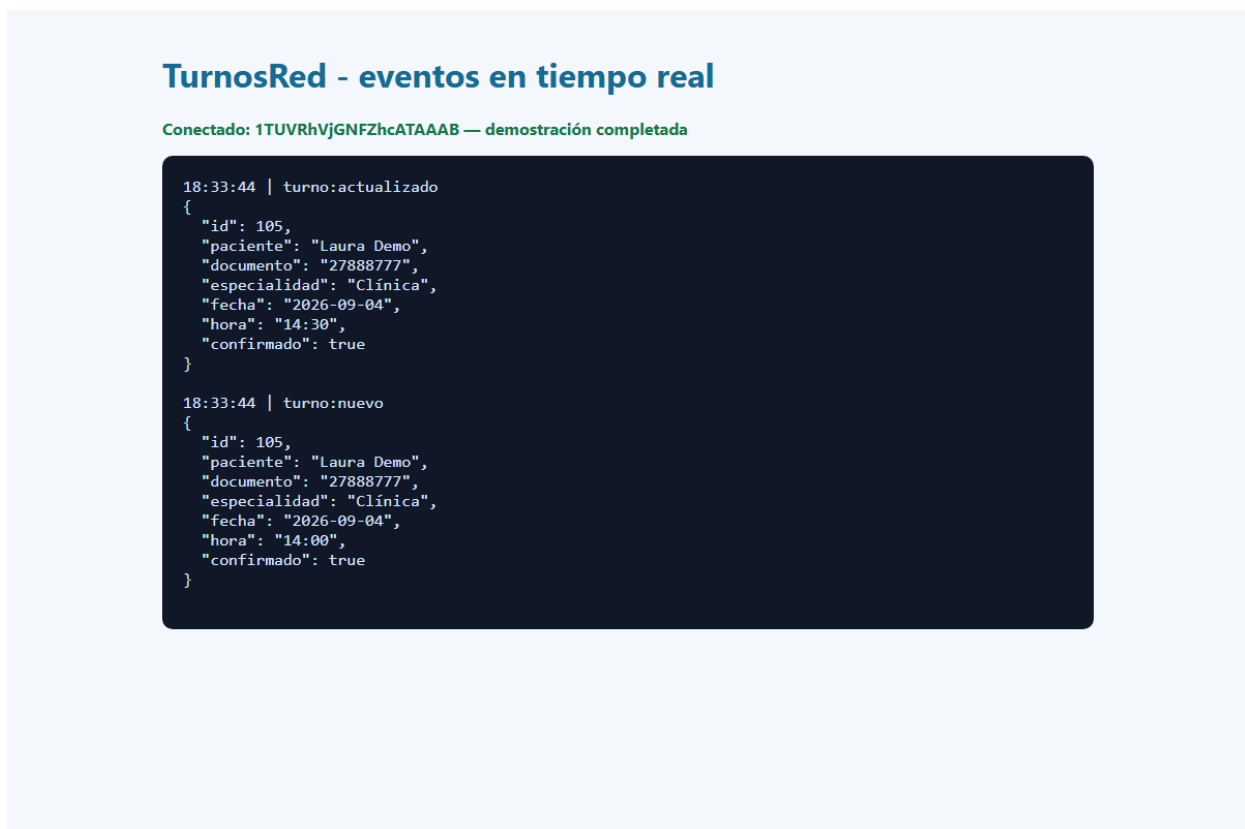
Captura 2. Ejecución reproducible de los cinco endpoints REST.

3. Eventos y comunicación en tiempo real

En las operaciones de alta, modificación y baja, el servicio emite un evento interno mediante EventEmitter. Socket.IO recibe esa notificación y la envía a los clientes conectados, por lo que la pantalla puede actualizarse sin recargas ni consultas periódicas.

Operación	Evento interno	Evento Socket.IO
POST	turno:creado	turno:nuevo
PUT	turno:actualizado	turno:actualizado
DELETE	turno:eliminado	turno:eliminado

La siguiente captura corresponde al cliente incluido en evidencias/cliente-socket.html. Se observa la recepción de un turno nuevo y su posterior actualización durante la misma conexión.



Captura 3. Eventos recibidos por Socket.IO sin recargar la página.

4. Cierre y entrega

El proyecto quedó publicado en GitHub con un historial progresivo de cambios. El README explica cómo instalarlo y ejecutarlo, mientras que la colección Postman, el cliente Socket.IO y los archivos JSON permiten repetir las comprobaciones.

Conclusión. La actividad me permitió integrar lectura asíncrona, tipado, validación, diseño REST y eventos en tiempo real dentro de una solución ordenada y verificable.